

Question Set - III



Maximum Marks = 60
Attempted Marks ≤ 65
Time Duration ≤ 2hrs

2 Marks Questions

Q1. Binary Tree In order Traversal (Recursively)

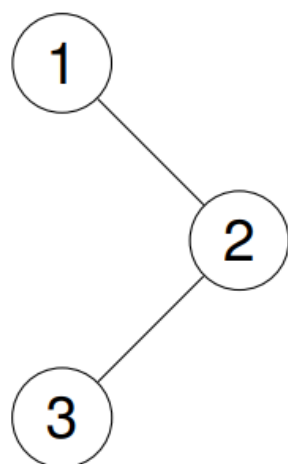
Given the `root` of a binary tree, return *the Inorder traversal of its nodes' values*.

Example 1:

Input: `root = [1,null,2,3]`

Output: `[1,3,2]`

Explanation:

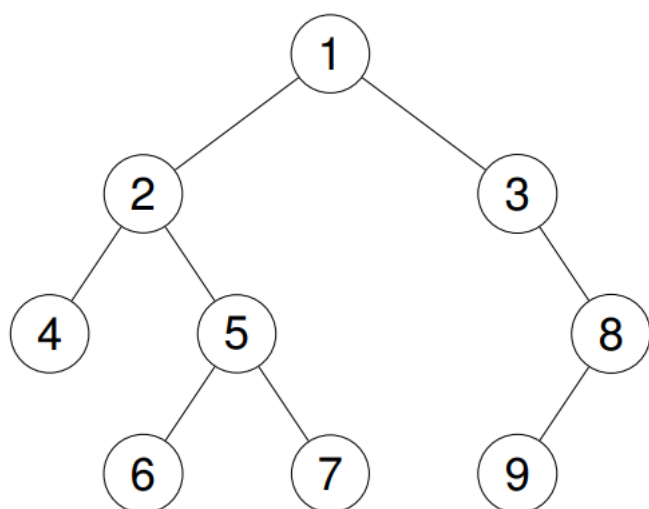


Example 2:

Input: `root = [1,2,3,4,5,null,8,null,null,6,7,9]`

Output: `[4,2,6,5,7,1,3,9,8]`

Explanation:



Example 3:

Input: `root = []`

Output: `[]`

Example 4:

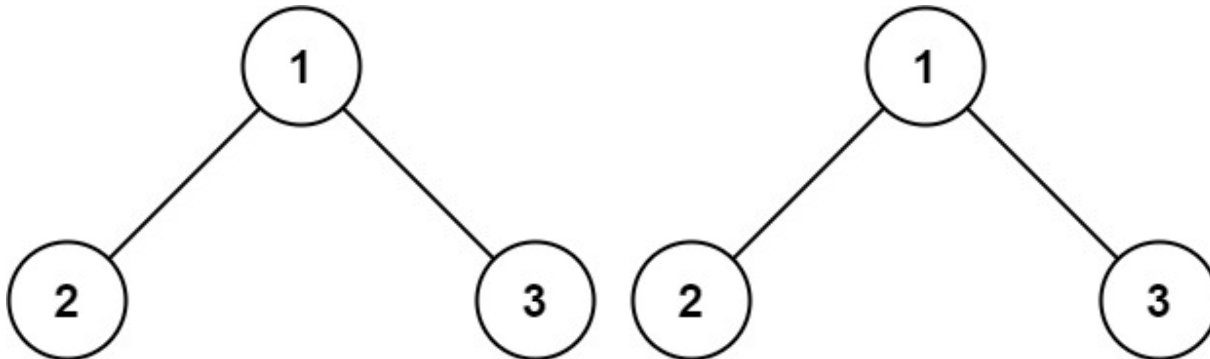
Input: `root = [1]`

Output: [1]

Q2. Same Tree

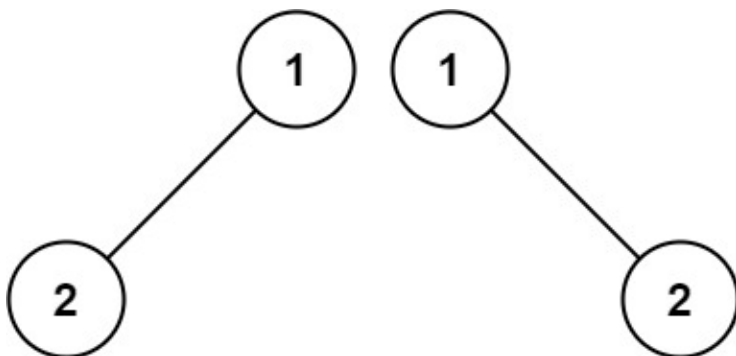
Given the roots of two binary trees `p` and `q`, write a function to check if they are the same or not. Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

Example 1:



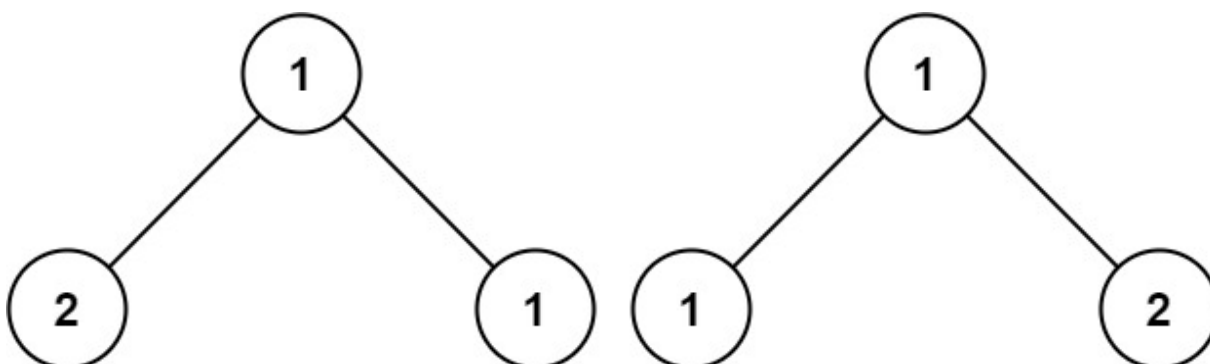
```
Input: p = [1,2,3], q = [1,2,3]
Output: true
```

Example 2:



```
Input: p = [1,2], q = [1,null,2]
Output: false
```

Example 3:

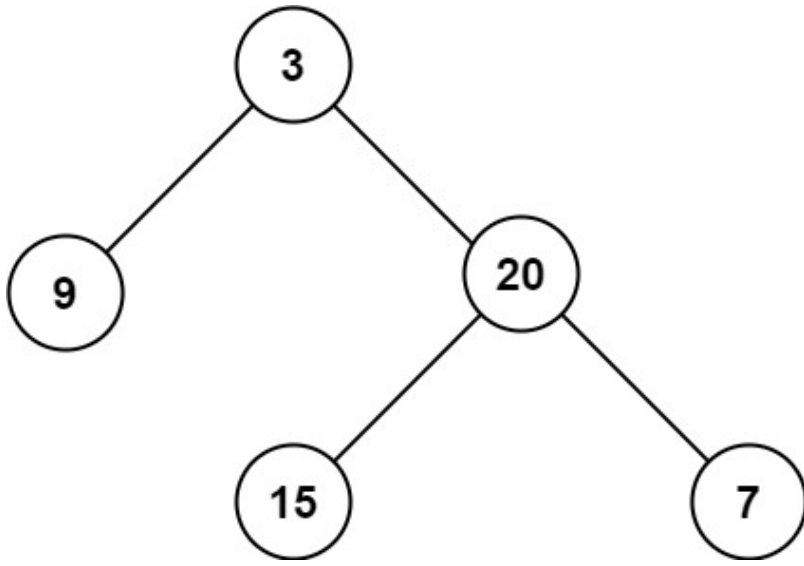


```
Input: p = [1,2,1], q = [1,1,2]
Output: false
```

Q3. Maximum Depth of Binary Tree

Given the `root` of a binary tree, return *its maximum depth*. A binary tree's **maximum depth** is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: 3

Example 2:

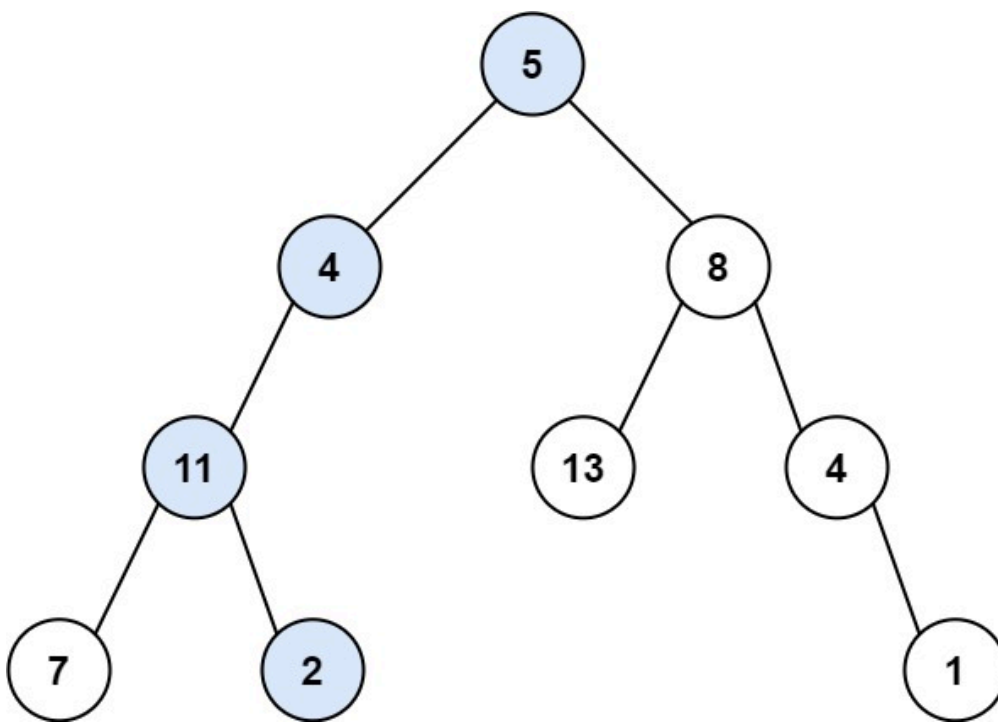
Input: root = [1,null,2]
Output: 2

Q4. Path Sum

Given the `root` of a binary tree and an integer `targetSum`, return `true` if the tree has a **root-to-leaf** path such that adding up all the values along the path equals `targetSum`.

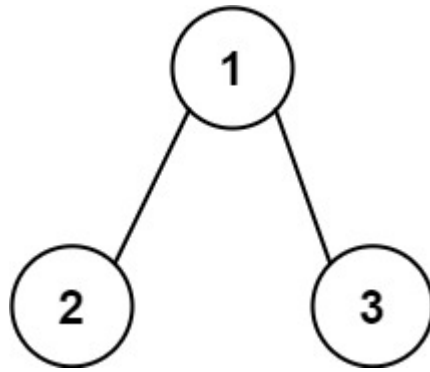
A **leaf** is a node with no children.

Example 1:



Input: root = [5,4,8,11,null,13,4,7,2,null,null,null,1], targetSum = 22
Output: true
Explanation: The root-to-leaf path with the target sum is shown.

Example 2:



```
Input: root = [1,2,3], targetSum = 5
Output: false
Explanation: There are two root-to-leaf paths in the tree:
(1 --> 2): The sum is 3.
(1 --> 3): The sum is 4.
There is no root-to-leaf path with sum = 5.
```

Example 3:

```
Input: root = [], targetSum = 0
Output: false
Explanation: Since the tree is empty, there are no root-to-leaf paths.
```

Q5. Binary Tree Pre order Traversal

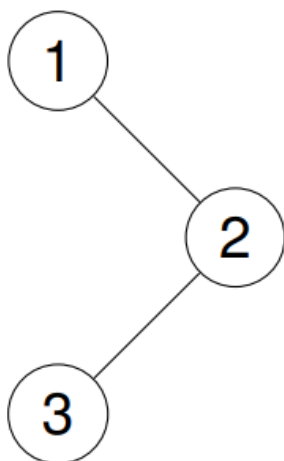
Given the `root` of a binary tree, return *the preorder traversal of its nodes' values*.

Example 1:

Input: root = [1,null,2,3]

Output: [1,2,3]

Explanation:

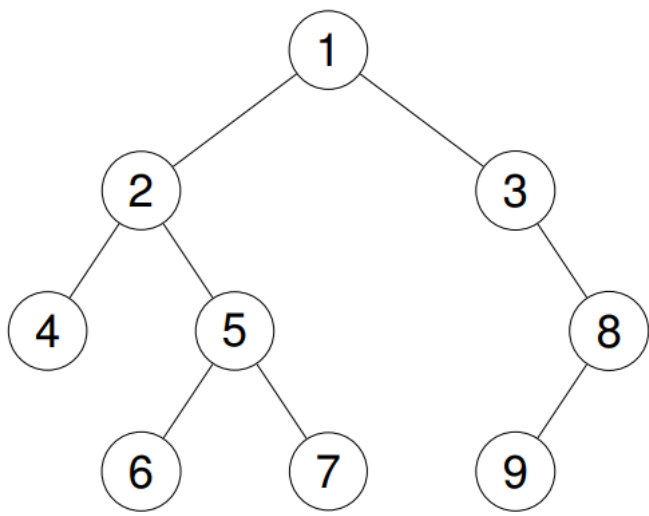


Example 2:

Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [1,2,4,5,6,7,3,8,9]

Explanation:



Example 3:

Input: root = []

Output: []

Example 4:

Input: root = [1]

Output: [1]

Q6. Binary Tree Post order Traversal

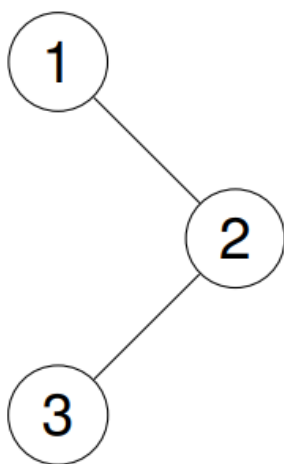
Given the **root** of a binary tree, return *the postorder traversal of its nodes' values*.

Example 1:

Input: root = [1,null,2,3]

Output: [3,2,1]

Explanation:

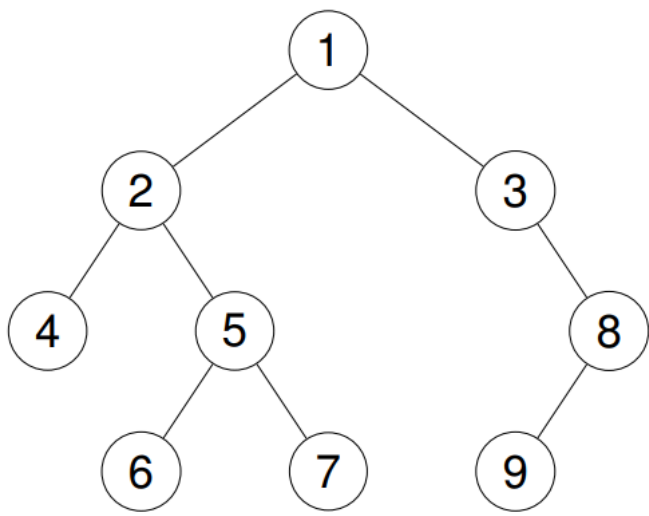


Example 2:

Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [4,6,7,5,2,9,8,3,1]

Explanation:



Example 3:

Input: root = []

Output: []

Example 4:

Input: root = [1]

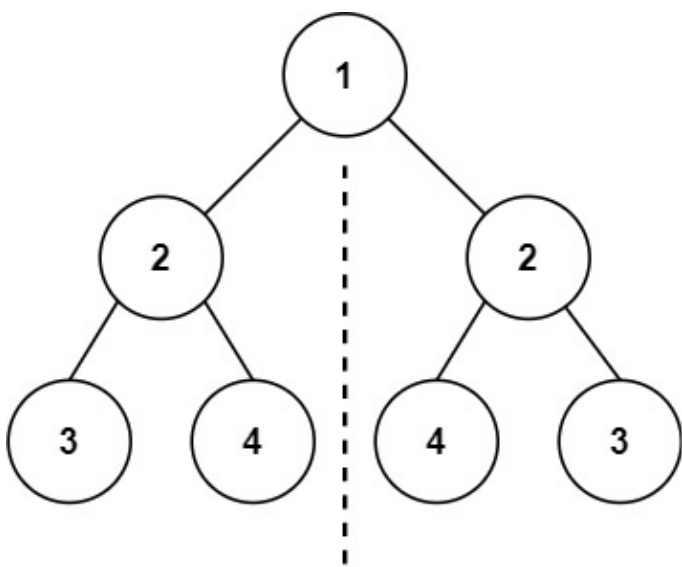
Output: [1]

3 Marks Questions

Q1. Symmetric Tree

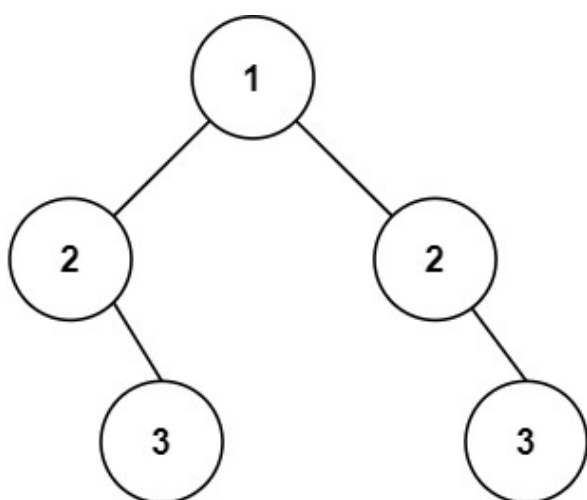
Given the **root** of a binary tree, *check whether it is a mirror of itself* (i.e., symmetric around its center).

Example 1:



Input: root = [1,2,2,3,4,4,3]
Output: true

Example 2:

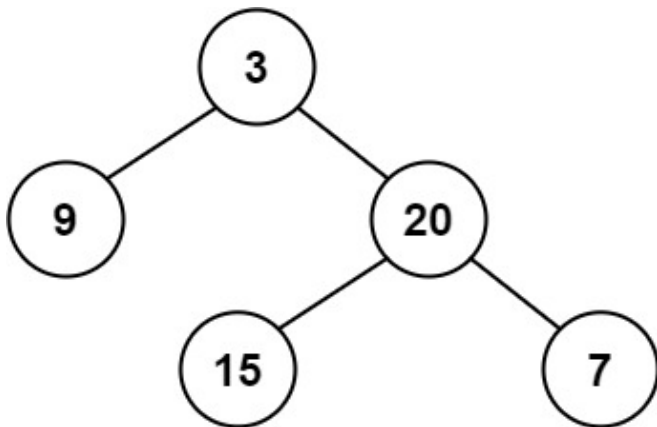


```
Input: root = [1,2,2,null,3,null,3]
Output: false
```

Q2. Balanced Binary Tree

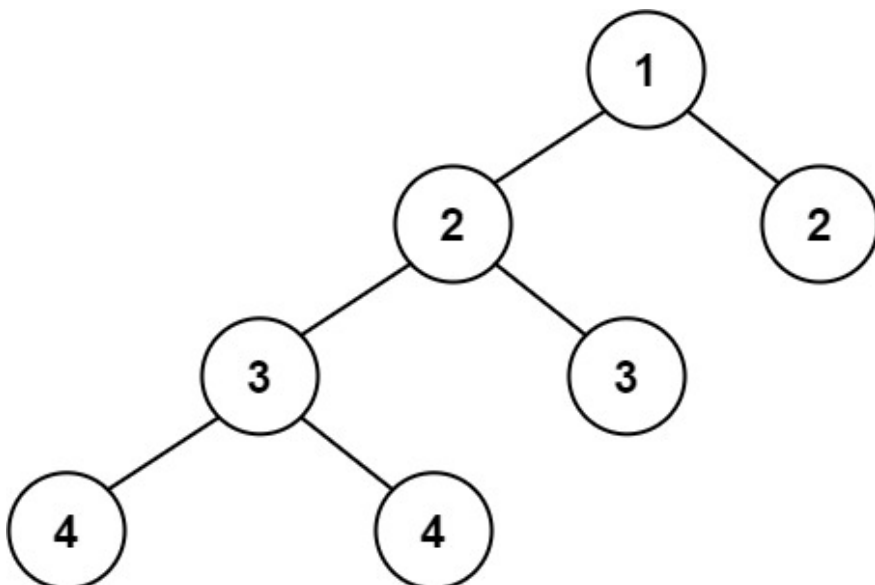
Given a binary tree, determine if it is **height-balanced**.

Example 1:



```
Input: root = [3,9,20,null,null,15,7]
Output: true
```

Example 2:



```
Input: root = [1,2,2,3,3,null,null,4,4]
Output: false
```

Example 3:

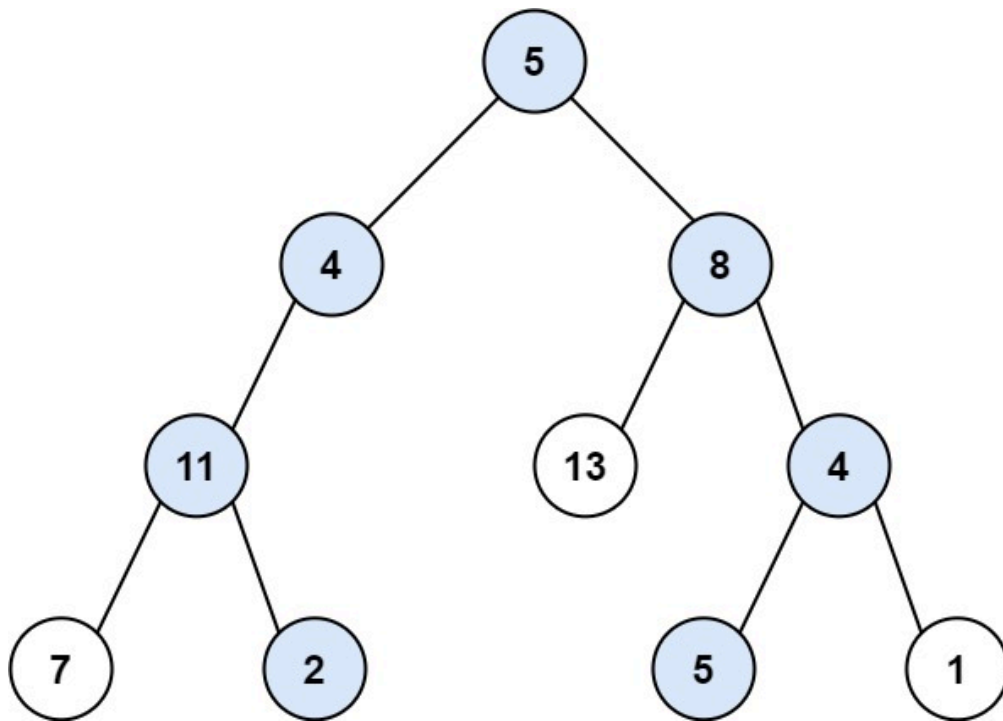
```
Input: root = []
Output: true
```

Q3. Path Sum II

Given the `root` of a binary tree and an integer `targetSum`, return *all **root-to-leaf** paths where the sum of the node values in the path equals `targetSum`*. Each path should be returned as a list of the node **values**, not node references.

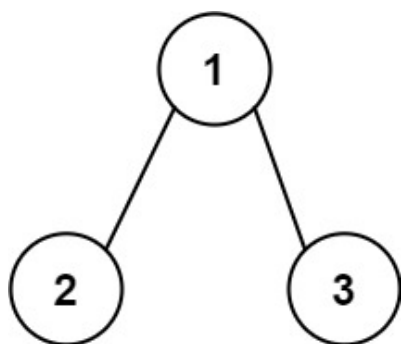
A **root-to-leaf** path is a path starting from the root and ending at any leaf node. A **leaf** is a node with no children.

Example 1:



Input: root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22
 Output: [[5,4,11,2],[5,8,4,5]]
 Explanation: There are two paths whose sum equals targetSum:
 $5 + 4 + 11 + 2 = 22$
 $5 + 8 + 4 + 5 = 22$

Example 2:



Input: root = [1,2,3], targetSum = 5
 Output: []

Example 3:

Input: root = [1,2], targetSum = 0
 Output: []

Q4. Validate Stack Sequences

Given two integer arrays `pushed` and `popped` each with distinct values, return `true` if this could have been the result of a sequence of push and pop operations on an initially empty stack, or `false` otherwise.

Example 1:

Input: pushed = [1,2,3,4,5], popped = [4,5,3,2,1]
 Output: true
 Explanation: We might do the following sequence:
 push(1), push(2), push(3), push(4),
 pop() -> 4,
 push(5),
 pop() -> 5, pop() -> 3, pop() -> 2, pop() -> 1

Example 2:

Input: pushed = [1,2,3,4,5], popped = [4,3,5,1,2]
 Output: false

Explanation: 1 cannot be popped before 2.

4 Marks Questions

Q1. Binary Tree In order Traversal (Iteratively)

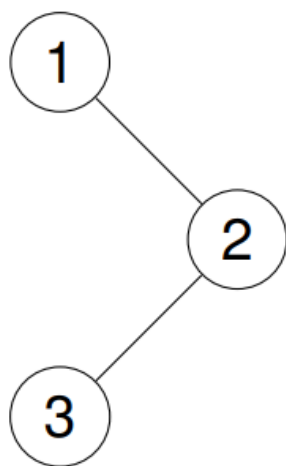
Given the `root` of a binary tree, return *the Inorder traversal of its nodes' values*.

Example 1:

Input: `root = [1,null,2,3]`

Output: `[1,3,2]`

Explanation:

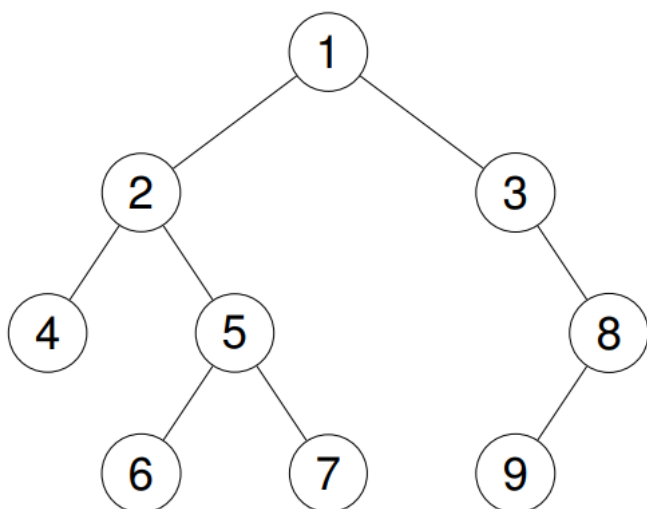


Example 2:

Input: `root = [1,2,3,4,5,null,8,null,null,6,7,9]`

Output: `[4,2,6,5,7,1,3,9,8]`

Explanation:



Example 3:

Input: `root = []`

Output: `[]`

Example 4:

Input: `root = [1]`

Output: `[1]`

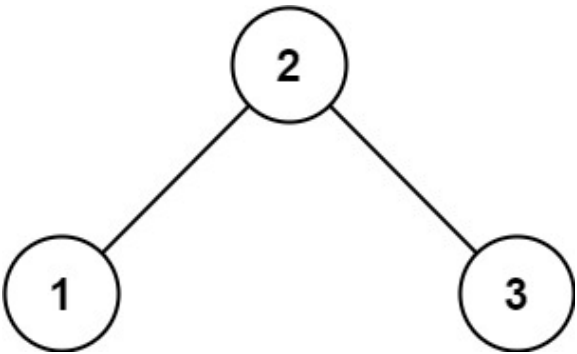
Q2. Validate Binary Search Tree

Given the `root` of a binary tree, *determine if it is a valid binary search tree (BST)*.

A **valid BST** is defined as follows:

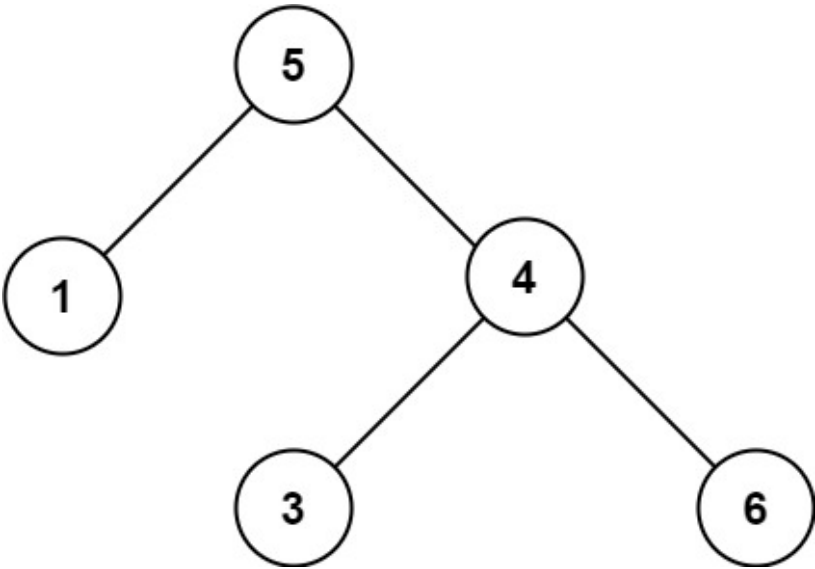
- The left of a node contains only nodes with keys **strictly less than** the node's key.
subtree
- The right subtree of a node contains only nodes with keys **strictly greater than** the node's key.
- Both the left and right subtrees must also be binary search trees.

Example 1:



```
Input: root = [2,1,3]
Output: true
```

Example 2:



```
Input: root = [5,1,4,null,null,3,6]
Output: false
Explanation: The root node's value is 5 but its right child's value is 4.
```

Q3. Valid Parentheses

Given a string `s` containing just the characters `'('`, `)'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: `s = "()"`

Output: `true`

Example 2:

Input: `s = "()[]{}"`

Output: `true`

Example 3:

Input: s = "[]"

Output: false

Example 4:

Input: s = "([])"

Output: true

Example 5:

Input: s = "([)]"

Output: false

Q4. Check If Word Is Valid After Substitutions

Given a string `s`, determine if it is **valid**.

A string `s` is **valid** if, starting with an empty string `t = ""`, you can **transform** `t` **into** `s` after performing the following operation **any number of times**:

- Insert string `"abc"` into any position in `t`. More formally, `t` becomes `tleft + "abc" + tright`, where `t == tleft + tright`. Note that `tleft` and `tright` may be **empty**.

Return `true` if `s` is a **valid** string, otherwise, return `false`.

Example 1:

```
Input: s = "aabcbc"
Output: true
Explanation:
"" -> "abc" -> "aabcbc"
Thus, "aabcbc" is valid.
```

Example 2:

```
Input: s = "abcabcababcc"
Output: true
Explanation:
"" -> "abc" -> "abcabc" -> "abcabcabc" -> "abcabcababcc"
Thus, "abcabcababcc" is valid.
```

Example 3:

```
Input: s = "abccba"
Output: false
Explanation: It is impossible to get "abccba" using the operation.
```

5 Marks Questions

Q1. Implement Stack Node

1. Create the constructor for the Stack Node
2. Create the Operation push
3. Create the Operation pop
4. Create the Operation isEmpty
5. Create the Operations peak and top

Q2. Implement Queue using Linked List

1. Create the Operation Enqueue
2. Create the Operation Dequeue

3. Create the Operation isEmpty
4. Create the Operation peek
5. Create the Operation display

Q3. Implement TreeNode

1. Create the constructor for the TreeNode
 2. Do traversal Recursively Preorder
 3. Do traversal Recursively Inorder
 4. Do traversal Recursively Postorder
-

10 Marks Questions

Q1. Implement Binary Search Tree

1. Create a method for insertion.
2. Create a method for searching.
3. Do traversal Recursively Inorder.